

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:50

Annexure A

Comparative Analysis

Code of Conduct:

I Ajay S - 192424078 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

AJAYS

Annexure A

Short Answer Test

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.
3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.
4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.
6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.
10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

- - - - -

Absolutely. Below are **assignment-ready answers** for Questions **2, 3, 4, 6, and 10**. I've kept each answer at a reasonable length so that the **complete set should fit within about 3 pages when handwritten**, while still covering the required comparison points.

2. Compare Locally Weighted Regression and Radial Basis Function Networks for Function Approximation

Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are machine learning techniques used to approximate complex functions. Both use the idea of locality, but they differ in how they learn and make predictions.

For example, consider a dataset containing values of x and $y = \sin(x)$. To predict the value of y for a new value of x , LWR gives more importance to training points that are closer to the query point. A weighted regression model is then fitted using these nearby points. Therefore, the model focuses mainly on the local region around the query.

An RBF network also uses local information, but it represents the function using several radial basis functions, usually Gaussian functions. Each hidden neuron responds strongly to points close to its centre. The final prediction is obtained by combining the outputs of these basis functions.

Comparison

- **Locality:** LWR calculates weights separately for every new query, making it highly local. RBF networks use fixed centres and widths after training, so locality is determined by the basis functions.
- **Computational Cost:** LWR can be computationally expensive during prediction because it performs a weighted regression for every new input. RBF networks generally have higher training complexity but faster prediction once trained.
- **Smoothness:** LWR can produce smooth predictions when an appropriate bandwidth is selected. RBF networks usually provide very smooth function approximations because Gaussian basis functions overlap.
- **Adaptability:** LWR easily adapts to new data because predictions are based directly on nearby training samples. RBF networks require retraining or updating their parameters when the underlying function changes significantly.

Thus, LWR is useful when local adaptability is important, whereas RBF networks are preferable when fast prediction and smooth function approximation are required.

3. Compare K-Nearest Neighbour and Radial Basis Function Models in Classification

K-Nearest Neighbour (KNN) and Radial Basis Function (RBF) models can both be used for classification, but they use different learning strategies.

Consider a dataset containing two classes of flowers based on petal length and petal width. KNN classifies a new flower by finding the K closest training samples and assigning the class that occurs most frequently among them. For example, with $K=5$, if four of the five nearest flowers belong to Class A, the new flower is classified as Class A.

An RBF classifier transforms the input using radial basis functions centred at selected training or prototype points. The outputs of these functions are then combined to determine the final class.

Comparison

- **Decision Boundaries:** KNN can create highly irregular and locally shaped decision boundaries because every prediction depends on nearby samples. RBF models generally produce smoother and more controlled nonlinear boundaries.
- **Training Time:** KNN has almost no actual training phase because it mainly stores the training dataset. RBF networks require training to determine centres, widths, and output weights.
- **Prediction Time:** KNN can be slower for large datasets because distances to many training points must be calculated. RBF models usually provide faster predictions after training.
- **Generalization:** KNN can perform well when similar examples belong to the same class, but it may be affected by noise and the choice of K. RBF networks can generalize better when their centres and widths are properly selected.
- **Memory:** KNN needs to store the complete training dataset, while an RBF model can represent the data using a smaller number of learned centres.

Therefore, KNN is simple and effective for smaller datasets, while RBF models are more suitable when smooth nonlinear decision boundaries and faster prediction are required.

4. Compare Case-Based Learning and Locally Weighted Regression Using a Recommendation System

Case-Based Learning (CBL) and Locally Weighted Regression (LWR) can both be applied to recommendation systems, but they handle new users and items differently.

For example, consider an online movie recommendation system. Suppose a new user has watched several movies similar to those watched by an existing user. In Case-Based Learning, the system searches its database for previous cases that are similar to the new user's preferences. It then recommends movies based on solutions or experiences associated with those similar cases.

In Locally Weighted Regression, user preferences and movie features can be represented numerically. The system gives greater weight to users or movies that are similar to the current query and uses these weighted examples to estimate the user's likely rating.

Comparison

- **Handling New Data:** CBL directly compares a new case with stored cases, so it can handle new situations without completely retraining a model. LWR also adapts quickly because predictions depend on nearby observations.
- **Adaptability:** CBL is highly adaptable because new cases can simply be added to the case database. LWR adapts by changing the weights assigned to nearby observations.
- **Computational Efficiency:** CBL may become computationally expensive when the case database becomes very large because many cases must be searched. LWR can

also be expensive at prediction time because a local regression calculation is performed for each query.

- Nature of Output: CBL is useful when previous cases provide direct examples of suitable recommendations. LWR is more suitable when the system needs to predict a continuous value such as a movie rating.
- Interpretability: CBL is relatively easy to explain because recommendations can be based on similar previous users or cases.

Thus, Case-Based Learning is useful for example-driven recommendation systems, while LWR is better when recommendations depend on predicting numerical preferences from nearby observations.

6. Compare Naïve Bayes and Logistic Regression for Spam Detection

Naïve Bayes (NB) and Logistic Regression (LR) are widely used for text classification tasks such as spam email detection. Both can handle high-dimensional text data efficiently, but their assumptions and learning approaches are different.

For example, an email containing words such as “*free*,” “*offer*,” “*winner*,” and “*prize*” may be classified as spam. Naïve Bayes calculates the probability that the email belongs to the spam class based on the words present. Logistic Regression learns weights for the words and calculates the probability of the email being spam.

Comparison

- Feature Independence: Naïve Bayes assumes that features are conditionally independent given the class. For text, this means it treats words as largely independent. This assumption is often unrealistic but works surprisingly well. Logistic Regression does not require this independence assumption.
- Probability Estimation: Naïve Bayes directly estimates class probabilities using Bayes' theorem. Logistic Regression estimates the probability of a class using a logistic function based on weighted features.
- Computational Efficiency: Naïve Bayes is extremely fast to train and requires relatively little computational power. Logistic Regression is also efficient, but training can require more computation, especially with large datasets.
- High-Dimensional Data: Both methods work well with text represented using features such as word counts or TF-IDF. Naïve Bayes is particularly effective when there are many features and relatively little training data.
- Performance: Logistic Regression can outperform Naïve Bayes when there are relationships between features that violate the independence assumption. Naïve Bayes may perform better when training data is limited and fast learning is required.

Therefore, Naïve Bayes is a good choice for fast and simple spam filtering, while Logistic Regression can provide better performance when sufficient training data is available and feature relationships are important.

10. Compare PCA and Feature Selection Techniques for Dimensionality Reduction

Principal Component Analysis (PCA) and Feature Selection are two important techniques used to reduce the dimensionality of a dataset. Both can improve model efficiency, but they achieve dimensionality reduction in different ways.

PCA transforms the original features into a smaller number of new variables called principal components. These components are combinations of the original features and are selected to preserve as much variance as possible.

Feature Selection, on the other hand, selects a subset of the original features and removes irrelevant or redundant features. Common methods include filter methods, wrapper methods, and embedded methods.

Comparison

- **Information Preservation:** PCA attempts to preserve the maximum amount of variance in fewer dimensions. Feature selection may lose some information if important features are removed.
- **Computational Complexity:** PCA requires calculations such as covariance matrices and eigenvectors or singular value decomposition, which can be computationally expensive for very large datasets. Simple feature-selection methods can be faster.
- **Interpretability:** Selected features remain in their original form, making feature selection highly interpretable. PCA components are combinations of several features and are therefore harder to interpret.
- **Model Performance:** PCA can improve performance when features are highly correlated and noisy. Feature selection can improve performance by removing irrelevant features while retaining meaningful information.
- **Downstream Tasks:** PCA is useful for visualization, clustering, and machine learning models affected by high dimensionality. Feature selection is particularly useful when interpretability and understanding the importance of original variables are important.

For example, in a dataset containing 100 features, PCA may reduce them to 10 principal components, while feature selection may directly select the 10 most useful original features.

Thus, PCA is preferable when information compression and correlation reduction are important, whereas Feature Selection is preferable when interpretability and retaining meaningful original features are the main goals.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand